Presentation Script — *The Wand Atelier*

Total target runtime: 15:00 (followed by 5:00 Q&A) Speaking pace assumption: ~150 words per minute → ~2,250 words total.

> ***Delivery notes:*** *- Bracketed `[...]` cues are stage directions, not spoken. - Slide transitions are flagged with ■ `NEXT SLIDE`. - Rehearse with a stopwatch; if you run long, trim slide 11 demo narration.*

Slide 1 — Title (0:00 – 0:30)

[Walk on. Wait for the room to settle. Make eye contact with the back row.]

Good morning. My name is Nayoung Lim, and the project I'm presenting today is called **The Wand Atelier**.

Before I tell you what it is, I want to ask you one question.

What if every NPC you met in a game had something they wouldn't tell you, and your job was to figure out what they actually needed?

That question is what I spent the last semester trying to answer.

■ NEXT SLIDE

Slide 2 — Abstract (0:30 – 1:30)

The Wand Atelier is a fantasy crafting game built in **Unity 6** where you play a wandmaker over a seven-day arc. Every day, customers walk into your shop with a problem. You read their dossier, buy materials from a market, craft a wand, and an AI judges how well your wand fits their needs.

■ NEXT SLIDE

What makes this project unusual is **how the content is made**. Three things in this game are not hand-authored. They are generated at runtime, every time you play.

First — **the customers**. Their names, professions, personalities, and the problems they bring you are written by **OpenAI's GPT-4o** the moment they walk in.

Second — **the materials**. Six fresh materials are generated every morning, each with their own pixel-art portrait drawn by a **local ComfyUI server** running a quantised image model on my laptop GPU.

Third — and this is the contribution I'm most proud of — **the evaluation**. The same language model that *invents* the customer also *judges* whether your wand fits them. The AI is both the storyteller and the critic.

The full game is around **eight thousand seven hundred lines of C#** across thirty-four scripts, eight scenes, and four branching endings. And the customers go through the deduction -> crafting -> generation loop.

■ NEXT SLIDE

Slide 3 — Introduction: The Problem (1:30 – 2:30)

So, why this project?

If you've ever played a crafting game — Stardew Valley, Don't Starve, Minecraft — you know the pattern. You learn the recipes. You memorise that iron plus coal makes steel. After a few hours, the system isn't really challenging you any more. You're just executing a lookup table you have in your head.

The same goes for NPCs. Most game characters say the same lines on every playthrough. The shopkeeper greets you the same way for the hundredth time.

The problem is — **the player ends up optimising *around* the game's systems, not *with* them**. The mystery dies the moment you crack the formula.

What I wanted to ask was: **what if logic, not lookup, decided the outcome?** What if every customer was different enough that no recipe could survive contact with them? And what if the only way to succeed was to *read carefully* and really carve out the needs of the random customers, not memorise?

And what if every wand you made gave you that thrill of discovery — fresh, every play, every craft?

■ **NEXT SLIDE**

Slide 4 — Objectives & Requirements (2:30 – 3:30)

That question gave me three project objectives.

[Gesture to slide.]

Objective one: Deduction over memorisation. I wanted reading and inference to matter more than rote learning. Customers should hide their real needs behind a surface request, and the player should have to dig those out.

Objective two: Creative flexibility. No fixed recipes. Any combination of materials should be valid if the player's reasoning is sound. The judge — the AI — has to evaluate intent, not match a key in a dictionary.

Objective three: Crafting as ritual. The act of making a wand should feel *embodied*. Not a button click — an actual skill check, with timing and attention.

To deliver on those, I set technical requirements: a seven-day game arc, four branching endings tied to player reputation, integration with two AI services running concurrently. Every requirement here was set to test whether the game can really use generative AI to make a fun game at production quality.

■ **NEXT SLIDE**

Slide 5 — Background & Literature Review (3:30 – 4:30)

I didn't invent any of these ideas in isolation. Four titles shaped this project.

[Point to influence map.]

Hogwarts Legacy showed me that gestural spell-casting — drawing a shape with the mouse — could feel like real magic. That became the tracing minigame.

Papers, Please showed me that deduction under time pressure is one of the most engaging mechanics in games. It's where the memo gate idea came from — you have to *commit* to an interpretation of a person, on the clock.

Disco Elysium showed me how much narrative density you can pack into a single character if you structure their description well. That gave me my seven-field customer schema.

And on the AI side — projects like **AI Dungeon** and **Inworld** demonstrated that LLMs can power believable NPCs.

But here's the gap. None of those projects actively use AI as the core mechanics of the game. My project tests if the new technology can provide a new game experience for players.

■ **NEXT SLIDE**

Slide 6 — The Gap This Project Fills (4:30 – 5:30)

Let me put that gap in a concrete table.

[Walk audience through the comparison left-to-right.]

Where existing crafting games use **recipe lookup**, my game uses **logic scoring** by an AI. Where existing games use **hand-written NPCs**, my game uses **LLM-generated dossiers** with a strict logical chain. Where existing games use **pre-rendered art**, my game uses **runtime pixel art** generated locally. Where existing puzzles have **one true answer**, my memo system uses a **soft gate** — your interpretation matters, even if it's wrong.

Why is any of this desirable? Three reasons.

One: infinite replayability without scaling content costs. The art and writing budget is essentially zero per playthrough.

Two: because the AI is both narrator and judge, the player gets feedback that's *coherent with the world*. The same voice that gave the customer their problem tells you whether you solved it.

Three: this is a viable production pattern for small studios. I'm running a quantised model on a laptop with eight gigabytes of VRAM. If a final-year student can do this, an indie studio absolutely can.

■ NEXT SLIDE

Slide 7 — Methodology Overview: Architecture (5:30 – 6:30)

Let's go technical.

[Point at the architecture diagram.]

The system has three components. The Unity client in the centre, OpenAI GPT-4o on the right, and ComfyUI running locally on the left.

GPT-4o handles all language work — about **six API calls per day in the game**. Customer generation, material generation, wand synthesis, and evaluation. Each call is a `UnityWebRequest` POST to the chat completions endpoint, parsed with Newtonsoft JSON.

ComfyUI handles **all image generation** — about seven images per day. I'm using a model called Z-Image Turbo with **Nunchaku INT4 quantisation**, which is the only way I can run a diffusion model on this hardware fast enough to be usable.

Cross-scene state lives in a singleton called `GameManager` — `DontDestroyOnLoad`, holds everything from the current customer to the player's gold, reputation, and which day it is.

Eight scenes total: title, morning, customer, market, crafting, the minigame, evaluation, and ending. Eight thousand seven hundred lines of C#. Thirty-four scripts. All flat — no subdirectories — to keep it simple.

■ NEXT SLIDE

Slide 8 — Methodology: The Customer Deduction Loop (6:30 – 7:30)

Let me walk you through one customer.

[Gesture to dossier screenshot.]

When a customer walks in, GPT-4o produces a **seven-field dossier** under a strict structural rule: **schoolOfMagic** → **profession** → **request** → **trueGoal** → **constraint**. Each field has to follow logically from the last.

Here's the design rule I gave the model — and I quote from my GDD — *"the constraint must be cruelest when it activates exactly when they need control most."*

So you might get a hydromancy field medic whose magic *amplifies* when she's emotionally distressed. The request says she needs precise pressure control. The true goal is to save more patients. The constraint is that the more lives are at stake, the more her magic spirals out.

Now — and this is the key mechanic — **the player can't carry the full dossier with them**. They have to read the prose and *click three words* into a memo card with three slots: Purpose, Personality, Element.

That memo — not the original dossier — is the only reference they have in the market.

A wrong memo *misleads* every choice that follows. There is no "correct" memo, just consequences.

■ **NEXT SLIDE**

Slide 9 — Methodology: Tracing Minigame (7:30 – 9:00)

After the player chooses materials, they enter the crafting ritual. This is the part that took me the longest to build.

[Point to minigame screenshots.]

It's three rounds. Each round, you trace a glowing rune path with your mouse. The trail fills **blue** as you trace it correctly. A **red mist** chases you from the start at a constant speed. If the red catches up before you finish, the round is lost.

But pure tracing got boring fast in playtests. So I added **gates** — interactive checkpoints along the path. Three types.

[Click through the gate icons.]

Tap gates ask you to press a displayed key once. **Hold** gates ask you to hold a key for nine-tenths of a second while a green ring fills. **Accent** gates ask you to press a key and *flick* the mouse in one of eight compass directions.

Round one is five tap gates. Round two adds a hold. Round three adds an accent. The complexity escalates.

The single most important design rule here: **the red mist never pauses while a gate is being resolved.** Every Hold and every Accent costs real time. The player can't stand still and think.

Technically, the path itself is built from six waypoints using **Catmull-Rom interpolation** for smoothness, and I wrote a custom **arc-length overlap validator** to guarantee the path never re-crosses itself in a way that would unfairly trap the cursor.

The final grade — A, B, C, or F — multiplies the reward by 1.0×, 0.85×, 0.7×, or 0.4×.

■ NEXT SLIDE

Slide 10 — Methodology: GPT-4o as Judge (9:00 – 10:30)

Now the most novel part of the project.

[Point to the prompt screenshot.]

When the player submits their wand, the same AI that *invented* the customer judges the wand. I send GPT-4o a structured prompt with the customer dossier, the wand name, and three attributes — and I ask for a JSON verdict.

[Read the rubric aloud, slowly.]

The scoring rubric is explicit: - If the wand only addresses the surface request, score forty to sixty. - If it addresses the customer's *true* goal, sixty to seventy-five. - If it addresses the true goal *and* accounts for the constraint, seventy-five to ninety-five. - If it creatively *resolves the tension* between true goal and constraint, ninety to one hundred.

The model returns a JSON object with `matchScore`, `whatWorked`, `whatMissed`, and a `customerReaction` line — verbatim dialogue from the imagined NPC.

That score plugs into the reward formula:

```
gold = 150 × (score / 100) × qualityMultiplier
```

If the score is below forty, you don't just earn nothing — you *lose* ten reputation. Bad wands have consequences.

■ NEXT SLIDE

Slide 11 — Results: Demo Walkthrough (10:30 – 12:00)

Let me show you what this looks like in motion.

[Start the demo OR play the pre-recorded clip. Narrate over it.]

This is **morning of day three**. The player gets a letter from their landlord — rent's due tonight.

Now we're in the customer scene. Notice the dossier on the left. Three slots are empty on the right. The player clicks **"war"** into Purpose, **"impatient"** into Personality, **"fire"** into Element.

[Pause for emphasis.]

The market — six materials, three cores and three woods. Notice the small **glyph** next to materials whose attributes match the memo. That's a hint, not a guarantee.

Now crafting. The player slots two cores and a wood. Confirm.

The minigame begins. Round one — easy, all taps. Round two — there's the hold gate, the player has to wait while the red mist closes in. Round three — accent gate, mouse flick, just barely makes it.

And finally — the evaluation reveal. The wand image appears, the score animates up, the AI's verdict prints itself out character by character: *"This blade-wood handle gives her precision, but the dragon-scale core may inflame her temper. A risky pairing."*

That verdict was *invented* in the same generation pass that gave us the customer. The narrative is closed-loop.

■ NEXT SLIDE

Slide 12 — Results: Evaluation & Critique (12:00 – 13:30)

Let me be honest about what worked and what didn't.

[Point to the 2×2 grid.]

Quantitatively — the build is around eight thousand seven hundred lines of code, runs at sixty FPS on a laptop, and the average GPT-4o latency is four to seven seconds, with image generation between eight and twelve seconds. The single most impactful UX decision in the whole project is the **parallel-execution pattern**: the wand image generates *while* the player traces the rune. This hides about seventy percent of the latency.

Qualitatively — playtesters reported two things consistently. The memo loop "feels like reading a real letter," and the Hold gates with the mist closing in were "stressful in a good way." That's the design objective being met.

Limitations — let me name three.

One: API cost. Each round costs me about four cents in OpenAI calls. That's not viable for free-to-play.

Two: the AI judge is non-deterministic. The same wand can score plus-or-minus ten across runs. I tried lowering temperature, but a strict-rubric system prompt only goes so far.

Three: the seven-day arc is tight. Players want more time to build a relationship with the world.

Validation — I ran a small A/B between random-prompt customers and structured-prompt customers. The structured ones produced about three times more "constraint-aware" wand submissions from playtesters. Small sample, but the signal is real.

■ NEXT SLIDE

Slide 13 — Conclusion & Future Work (13:30 – 14:30)

[Slow down. This is your closing.]

The contribution of this project is a working demonstration that games *can* use generative AI for both content and evaluation in a real-time game, and that careful UX — specifically parallel execution — can hide the latency cost.

What's done: the seven-day arc, four endings, the memo gate, the three-gate minigame, the dual-AI pipeline, the parallel-execution pattern, and a near-complete UI Toolkit migration.

What I'd build in the next six months: two customers per day, a commission system that pays a one-point-seven-five times multiplier for special orders, reputation-tier customer biasing — high-rep players see royalty, low-rep players see brigands — a deterministic scoring rubric to fix the non-determinism problem.

Long-term — procedural rune shapes for the minigame, multi-day customer threads where the same NPC returns, and a modder API so the community can extend the prompt library.

■ NEXT SLIDE

Slide 14 — Appendices, References, Q&A Invitation (14:30 – 15:00)

[Gesture to the slide.]

The full software stack is on this slide — Unity 6, URP, GPT-4o, ComfyUI with the Nunchaku INT4 model.

The full code is in the GitHub repository linked here. Thirty-four scripts, eight thousand seven hundred lines of C#.

References to the influences I named earlier are at the bottom — Hogwarts Legacy, Papers Please, Disco Elysium, plus the Catmull-Rom and Nunchaku papers.

[Pause. Make eye contact.]

Thank you very much for your time. I'm happy to take questions.

[Step back from the lectern. Wait for the moderator.]

Time-trim notes (in case you run long)

If you reach **slide 11 at 10:50 or later**, cut the demo narration to: - Just morning + memo + minigame + verdict (skip market commentary).

If you reach **slide 12 at 12:30 or later**, drop the A/B validation paragraph entirely. The qualitative quotes carry the slide on their own.

If you reach **slide 13 at 14:00 or later**, deliver only the *Done* column and skip *Long term*.

Time-pad notes (in case you run short)

If you reach **slide 14 at 14:30**, add: *"I'd also like to acknowledge my supervisor and the playtesters who broke the build in creative ways — their feedback shaped most of the design decisions you saw today."*